

Project Overview & Status Report

Multi-Modal IoT & Deep Learning Framework for Early Prediction of Sepsis in Smart ICU Environments

Final-year B.E. CSE project | Written for a non-technical audience

1. What Problem Are We Solving?

Sepsis is a life-threatening condition where the body's response to an infection starts damaging its own organs. It can turn fatal within hours, but caught early it is very treatable. The problem is that early warning signs (small changes in heart rate, temperature, blood pressure, breathing rate) are easy for a busy nursing staff to miss, especially when watching many patients at once.

This project builds a computer system that watches a patient's vital signs continuously (the same way a hospital monitor would), and uses Artificial Intelligence to calculate a running "risk score" for sepsis. If that score gets dangerously high, the system immediately raises a clear, impossible-to-miss alert for the clinical staff, and explains, in plain terms, which vital signs are driving the concern.

2. How The System Works, Step By Step

Think of the system as a pipeline. Data flows from the patient's bedside all the way to the doctor's screen in a few seconds:



- **Step 1 -- Collecting vitals:** Heart rate, oxygen level, temperature, blood pressure and breathing rate are read continuously. Today this is a realistic simulator that behaves exactly like real hospital sensor hardware would (so swapping in real sensors later needs no changes anywhere else in the system).
- **Step 2 -- Sending the data live:** Every reading is streamed in real time over a lightweight, industry-standard messaging protocol (the same kind of technology used in IoT devices worldwide), so the system reacts within seconds of a change, not minutes.
- **Step 3 -- The AI reads the pattern:** A deep-learning model -- trained on thousands of real ICU patient records -- looks at the recent trend of vitals (not just one reading) and recognises patterns that historically led to sepsis.
- **Step 4 -- A risk score is calculated:** The AI outputs a probability (e.g. "78% risk") which is then translated into a simple label anyone can understand: **Low, Medium, High, or Critical.**
- **Step 5 -- An alert is raised if needed:** For High or Critical risk, the system immediately shows a red pop-up alert on every screen the clinician has open, explaining *why* (e.g. "Heart rate elevated at 128 bpm, Temperature elevated at 39.4°C") -- not just a number, but the reasoning behind it.
- **Step 6 -- The doctor decides:** The clinician sees the full picture on a live dashboard (charts, trends, AI explanation) and can download a professional, hospital-style PDF report to print, archive, or hand off to another doctor.

3. What Has Been Built So Far

The project is organised into 12 planned steps. All 12 are complete for the software side of the system (everything that does not require physical sensor hardware). Physical hardware integration is a separate, not-yet-started phase, explained in Section 6.

#	What was built	In plain words	Status
1	Blueprint & Design	Planned out the whole system on paper first: how data flows, how the database is organised, and the exact data format any sensor (real or simulated) must send.	DONE
2	Real Hospital Data	Loaded two well-known, publicly available real ICU patient datasets (PhysioNet and MIMIC-IV) and cleaned/organised them so the AI could learn from real cases.	DONE
3	Data Exploration	Studied the data in depth -- which vital signs matter most, what's missing, what's normal vs. abnormal -- before building anything, so the AI is built on solid understanding.	DONE
4	AI Model Design	Designed the "brain" of the system: a deep-learning model that reads a sequence of vitals over time and understands trends, not just single snapshots.	DONE
5	AI Model Training	Trained that AI model on the real hospital data, tuning it carefully so it learns genuine warning patterns rather than memorising the data.	DONE
6	Explainable AI	Added a layer that makes the AI show its reasoning -- which specific vital signs pushed the risk score up or down -- so results aren't a "black box."	DONE
7	Secure Database	Built a proper database to permanently and safely store every patient, every vital reading, every AI prediction, and every alert, with built-in data-quality checks.	DONE
8	Backend Engine	Built the server that all other parts talk to -- it handles every request (add a patient, fetch vitals, run a prediction) securely and consistently.	DONE
9	Live Dashboard	Built the premium, hospital-themed web dashboard clinicians actually use: login, patient list, live monitoring charts, AI explanations, and an admin panel.	DONE
10	Alert System	Built the engine that watches every new AI result and automatically raises an alert (with cooldown rules to avoid spamming) whenever risk crosses a dangerous threshold.	DONE
11	Real-Time Streaming	Connected everything end to end so vitals flow in live, get analysed by the AI, and appear on the dashboard within seconds -- proven with a realistic simulated data stream.	DONE

#	What was built	In plain words	Status
12	Full Integration & Testing	Wired every piece together and tested the entire system as a whole, from a vital-sign reading arriving to a doctor seeing an alert and downloading a report.	DONE

4. Key Features, In Detail

Beyond the core AI pipeline, several client-facing features have been added recently:

- **Premium, hospital-grade dashboard:** A dark, modern clinical interface (not a generic admin template) with live charts, colour-coded risk badges, and a dedicated Admin panel showing system health and every user action.
- **Real-time red-alert pop-ups:** The moment the AI flags a patient as High or Critical risk, a red alert pops up on screen -- with a siren icon and a plain-English reason -- no matter which page the clinician is on.
- **Downloadable clinical PDF reports:** One click produces a formatted, hospital-branded PDF for any patient: current risk, vitals history, AI trend, explanation, and full alert history -- ready to print or archive.
- **Private accounts for every clinician/admin:** Each login only ever sees the patients it admitted. A brand new clinician account starts completely empty -- no old test data, no other clinician's patients -- exactly like separate real hospital accounts should behave.
- **Self-service "Admit Patient":** Any clinician or admin can add a new patient themselves directly from the dashboard, rather than relying on someone else to seed data for them.
- **Full audit trail:** Every login, every alert raised, every alert acknowledged, and every patient admitted is permanently logged -- so there is always a record of who did what and when, which is essential for a healthcare-style system.

5. Technology Used (In Plain Terms)

Technology	What it actually does here
Python	The programming language the entire system (AI, server, dashboard) is written in.
TensorFlow / Keras	The toolkit used to build and train the AI model that predicts sepsis risk.
Deep Learning Model (CNN + LSTM + Transformer)	The AI "brain" itself -- a model specifically good at spotting patterns in data that changes over time, like vital signs.
SHAP	The "explainability" tool that lets the AI show which vital signs influenced its decision, and by how much.
MongoDB	The database that permanently and safely stores every patient, reading, prediction and alert.
FastAPI	The backend engine/server that every other part of the system talks to, in a secure, organised way.
MQTT (Mosquitto)	The live-messaging technology used to stream sensor readings in real time -- the same standard used across real-world IoT devices.
Plotly Dash	The framework used to build the live, interactive web dashboard clinicians use.
ReportLab	The tool used to generate the formatted, hospital-style PDF clinical reports.
JWT (secure login tokens)	The technology that keeps every login secure and makes sure each account only ever sees its own patients.
Docker	Used to run supporting services (the database, the messaging system) in a consistent, easy-to-set-up way.
Pytest (testing)	The tool used to automatically re-check that every part of the system still works correctly after every change.

6. How Do We Know It Works?

The system isn't just "built" -- it has been actively tested at every level:

- **300+ automated tests** run against the real database and real server on every change, covering everything from "does login work" to "does a full patient journey from admission to alert to PDF report work end to end."
- **Live, real demonstrations were run** -- not just automated tests -- including creating a brand-new clinician account and confirming it genuinely sees zero other patients, and manually triggering High/Critical predictions to confirm the red alert really appears with the correct reasoning.
- **Model accuracy is measured honestly:** on held-out real patient data the AI achieves an AUROC of 0.766 (a standard accuracy measure where 0.5 is random guessing and 1.0 is perfect) -- meaningfully better than chance, and its risk ranking (Low/Medium/High/Critical) has been checked against real patient trajectories.

An honest limitation, stated plainly:

Sepsis is rare in the training data used (under 2% of records), which makes it a genuinely hard prediction problem, and the current model is a single model (not a large ensemble) trained on limited compute. This means it is meaningfully useful as a research and educational system, and its risk stratification is directionally correct, but it is **not yet a clinically validated medical device** and must not be used as the sole basis for a real clinical decision. This caveat is deliberately shown on every generated PDF report as well.

7. What's Left To Do

The entire software system described above (Steps 1-12) is complete and working end to end. What remains is everything that requires either physical hardware or a level of validation beyond a student project's scope:

#	What's needed	In plain words	Status
A	Real Sensor Hardware	Connect actual physical sensors (ESP32 microcontroller + real heart-rate/temperature/etc. sensors) at a real bedside, instead of the current realistic simulator. The system was deliberately designed so this requires zero changes to the AI, database, or dashboard -- only the sensor device changes.	PLANNED
B	Larger, More Balanced Training Data	Sepsis cases are rare in the current dataset. Training on more data (or combining several hospitals' data) would likely improve the AI's accuracy further, especially at catching more of the true sepsis cases.	PLANNED
C	Clinical Validation	Before any real hospital use, the system would need to be reviewed and validated by qualified medical professionals, and likely go through a formal clinical trial process -- standard for any medical decision-support tool.	PLANNED
D	Notifications Beyond the Dashboard	Currently alerts appear on-screen; SMS, mobile push notifications, or a dedicated mobile app for on-call staff would extend reach beyond someone actively watching the dashboard.	PLANNED
E	Production Hardening	Before any live deployment beyond a demo/academic setting: a formal security review, load-testing for many simultaneous patients, and a professional deployment (cloud hosting, backups, monitoring) would be needed.	PLANNED

8. Summary

In short: the full software pipeline -- from collecting live vital signs, through an AI model that predicts sepsis risk and explains its reasoning, to real-time red alerts and downloadable clinical PDF reports, all wrapped in a secure, multi-account dashboard -- is built, tested, and working today with realistic simulated sensor data. What remains is connecting real physical hardware, growing the training data, and the formal clinical validation any tool would need before real hospital use.

This document is a plain-language summary for review purposes. Technical documentation (architecture, database design, API reference, and full test reports) is maintained separately in the project's own documentation folder.